

Phase 1B: Forensic Write Operations & Master Overwrite Audit Report

Mutation Inventory & Security Vulnerability Assessment for Postgres Migration

TARGET SYSTEM	AUDIT SCOPE	MASTER OVERWRITE RISK	TARGET SECURITY LAYER
120+ Active Branch Terminals	All Firestore Mutations (SDK + REST)	CONFIRMED CRITICAL	PostgreSQL Row-Level Security (RLS)

CRITICAL AUDIT CONFIRMATION: Forensic analysis confirms that **regular branch terminal operators currently CAN and DO overwrite bank-wide master settings** (including `settings/rulesMaster`, `settings/branchSeeds`, `settings/valuersList`, and `settings/productsList`). This occurs both through unvalidated UI actions and automated 5-second background synchronization fallbacks.

1. Root Cause Analysis: How Branch Terminals Overwrite Master Settings

There are three concurrent mechanisms in the codebase enabling branch operators to mutate or overwrite bank-wide configuration documents:

A. Automated 5-Second Background Sync Fallbacks (`syncCloudData`)

In `app_gold.js` (lines 946-964), the automated sync timer executes every 5,000ms on all 120+ branch machines. If a fetch for bank rules or settings momentarily returns empty or times out, the branch machine immediately pushes its local `localStorage` state back to the cloud:

```
// app_gold.js (Lines 947-964) if (typeof window.FirebaseService.getRules === "function") { const fbRules = await window.FirebaseService.getRules(); if (fbRules && typeof fbRules === "object" && fbRules.membership) { state.rules = { ...state.rules, ...fbRules }; } else if (state.rules && state.rules.membership) { // ⚠️ ANY branch terminal pushes its local rules to Firestore if fetch is empty! window.FirebaseService.saveRules(state.rules).catch(() => { }); } }
```

B. Continuous Unconditional Valuer Master Writes

In `app_gold.js` (line 1031), `window.FirebaseService.saveValuersList(state.valuers, state.deletedValuerIds)` is called **unconditionally on every single 5-second sync cycle from every single connected terminal**, causing continuous concurrent write race conditions across 120+ machines.

C. Missing Role Checks on Master Setting Forms & Zero Server-Side Rules

- UI Handlers:** In `app_gold.js` (lines 7200-7214), the Rules Master save form does not verify if `state.currentSession.code === "99"`. Any branch operator can edit deduction percentages or LTV rules and commit them directly.
- Firestore Rules:** `firestore.rules` contains `allow read, write: if true;`, providing zero database-level authorization.

2. Complete Inventory of All Firestore Write Operations

The table below documents every single database mutation point across `firebase-config.js`, `app_gold.js`, and `management.js`:

OPERATION	TARGET RESOURCE	SOURCE FILE & LINE	METHOD	SCOPE	TRIGGER & AUTHORIZATION
saveLoan	<code>loans/{loanId}</code>	<code>firebase-config.js:470</code> <code>app_gold.js:3634</code> (New) <code>app_gold.js:4167</code> (Edit) <code>app_gold.js:9896</code> (Status) <code>app_gold.js:10041</code> (Close) <code>app_gold.js:10219</code> (Renew) <code>app_gold.js:1092</code> (Sync)	SDK <code>doc.set(merge)</code> + REST PATCH	Branch Loan Record (contains <code>branchId</code>)	User action (Loan CRUD) + 5s auto- sync for offline loans. Open in DB
deleteLoan	<code>loans/{loanId}</code> + <code>deleted_loans/{id}</code>	<code>firebase-config.js:569,</code> 576 <code>app_gold.js:4034</code> (Rename) <code>app_gold.js:4436</code> (Delete)	SDK <code>delete()</code> + SDK <code>set()</code> + REST DELETE/PATCH	Target Loan Doc	User action (Loan Delete Modal). No branch ownership check
saveDailyRates	<code>rates/today</code> + <code>settings/dailyRates</code>	<code>firebase-config.js:717,</code> 718 <code>app_gold.js:2161,</code> 2196, 2286	SDK <code>doc.set(merge)</code> + REST PATCH	 GLOBAL BANK- WIDE	Daily rate entry / Rate lock modal. UI HO check only; DB open
saveRules	<code>settings/rulesMaster</code>	<code>firebase-config.js:910</code> <code>app_gold.js:962</code> (5s Sync) <code>app_gold.js:7211</code> (Form) <code>app_gold.js:7232</code> (Reset) <code>app_gold.js:7335,</code> 7488, 7530	SDK <code>doc.set(merge)</code> + REST PATCH	 GLOBAL BANK- WIDE	5s Auto-Sync fallback + Rules Master form / custom charge toggle. Branch PCs can overwrite
saveSettings	<code>settings/branchSeeds</code>	<code>firebase-config.js:840</code> <code>app_gold.js:952</code> (5s Sync) <code>app_gold.js:7609,</code> 7724	SDK <code>doc.set(merge)</code> + REST PATCH	 GLOBAL BANK- WIDE	5s Auto-Sync fallback + Settings form submission. Branch PCs can overwrite
saveValuersList	<code>settings/valuersList</code>	<code>firebase-config.js:1039</code> <code>app_gold.js:1031</code> (5s Sync) <code>app_gold.js:6303,</code> 6418, 6533	SDK <code>doc.set()</code> + REST PATCH	 GLOBAL BANK- WIDE	Continuous 5s Loop from ALL 120+ PCs + Valuer CRUD modal. Continuous overwrite

OPERATION	TARGET RESOURCE	SOURCE FILE & LINE	METHOD	SCOPE	TRIGGER & AUTHORIZATION
saveProductsList	settings/productsList	firebase-config.js:1099 app_gold.js:1041 (5s Sync) app_gold.js:6604, 6692	SDK <code>doc.set(merge)</code> + REST PATCH	 GLOBAL BANK-WIDE	5s Auto-Sync fallback + Loan scheme CRUD modal. Branch PCs can overwrite
saveBranchesList	settings/branchesList	firebase-config.js:978 app_gold.js:1518, 6079	SDK <code>doc.set(merge)</code> + REST PATCH	 GLOBAL BANK-WIDE	Branch Master edit / Password change. DB open
saveBranch	branches/{branchCode}	firebase-config.js:265 app_gold.js:1506, 6069	SDK <code>doc.set(merge)</code>	Single Branch Doc	Branch Master modal save. DB open
deleteBranch	branches/{branchCode}	firebase-config.js:274	SDK <code>doc.delete()</code>	Single Branch Doc	Branch deletion. DB open
toggleBranchStatus	branches/{branchCode}	firebase-config.js:282	SDK <code>doc.update()</code>	Single Branch Doc	Active/Inactive toggle. DB open
seedDefaultBranches	branches/{branchCode}	firebase-config.js:240	SDK <code>batch.set(merge)</code>	All 17 Branches	Cold start initial seed. DB open
saveUserRole	users/{uid}	firebase-config.js:316	SDK <code>doc.set(merge)</code>	User RBAC Doc	Admin User Management. DB open
saveCustomer	customers/{custId}	firebase-config.js:780 app_gold.js:3639, 6845	SDK <code>doc.set(merge)</code>	Customer Record	Loan Submit / Customer Directory CRUD. Global customer pool
deleteCustomer	customers/{custId}	firebase-config.js:822 app_gold.js:6980	SDK <code>doc.delete()</code>	Customer Record	Customer Directory Delete. Global customer pool
updateDeviceHeartbeat	active_sessions/{sid}	firebase-config.js:1310 app_gold.js:1282, 1309, 1332	SDK <code>doc.set(merge)</code> + REST PATCH	Per-Device Session	Login + Continuous 30s background timer. Device session write
terminateActiveSession	active_sessions/{sid}	firebase-config.js:1343 management.js:372	SDK <code>doc.set(merge)</code> + REST PATCH	Target Device Session	Remote Killswitch Disconnect from Management. Admin Action
deleteActiveSession	active_sessions/{sid}	firebase-config.js:1403 app_gold.js:1363 management.js:399	SDK <code>delete()</code> + REST DELETE	Target Device Session	User Logout / Session cleanup. Device session
logAuditEvent	audit_logs/{logId}	firebase-config.js:1199 app_gold.js:1275, 1357 management.js:81, 374	SDK <code>doc.set()</code> + REST PATCH	Audit Record	Security audit event tracking. Not immutable in Firestore

3. Security Architecture Blueprint for PostgreSQL & Supabase

The table below outlines how every write vulnerability identified above will be permanently resolved at the database engine level in PostgreSQL using Row-Level Security (RLS) and constraints:

RESOURCE / TABLE	CURRENT VULNERABILITY IN FIRESTORE	POSTGRESQL & SUPABASE SECURITY ARCHITECTURE
Master Rules & Settings (rules_master , bank_settings , products , valuers)	Branch terminals overwrite bank-wide settings via 5s sync loop and unvalidated forms.	Postgres RLS Policy: FOR SELECT TO authenticated USING (true); FOR ALL TO authenticated USING (auth.jwt() ->> 'role' = 'admin' AND auth.jwt() ->> 'branch_id' = '99'); <i>Branch users receive a hard SQL 403 Forbidden on any write attempt.</i>
Loan Applications (loans , loan_ornaments)	Any branch can modify or delete loans belonging to other branches.	Branch-Scoped RLS: FOR ALL TO authenticated USING (auth.jwt() ->> 'role' = 'admin' OR branch_id = (auth.jwt() ->> 'branch_id')::text); <i>Branch users can only insert, update, or delete loans within their assigned branch.</i>
Security Audit Trail (audit_logs)	Audit logs can be modified or deleted by clients.	Immutable Append-Only Table: FOR INSERT TO authenticated WITH CHECK (true); FOR UPDATE, DELETE TO authenticated USING (false); <i>No user, including admins, can modify or delete historical audit entries.</i>
Daily Gold Rates (rates)	Unprotected daily rate updates.	Locked Rates Enforcement: Only HO Super Admin can insert/update rates. If is_locked = true , a database trigger rejects further updates for that date.